

Chapter 2

Designing the Multi-Stage Recsys Architecture

2.1 Introduction: The End-to-End Framework

This chapter is the design & architectural backbone of the book. Before diving into specific models, we need a mental framework for how all the pieces fit together. If Chapter 1 was about “Why RecSys matters,” this chapter is about “how to think through building one”.

There is an important distinction to draw upfront: the project lifecycle and the system architecture are not the same thing, though they are deeply interrelated.

The **project lifecycle** describes the sequence of decisions and activities you go through when building a RecSys initiative: defining the business problem, choosing metrics, collecting data, engineering features, training models, deploying, and iterating. It is a temporal process with phases that roughly proceed in order (though in real life, you will loop back frequently as you discover new information).

On the other hand, the **ML system architecture** describes the runtime components of your production recommender system: the retrieval layer, the ranking layer, the re-ranking and policy layer, the feature store, the serving infrastructure. These components run simultaneously in production, processing requests in real time. They also need to be supported offline for example, the vector store often used in the retrieval layer will need an indexing strategy for the item embeddings, etc. The design choices for architecting the system are usually very engineering heavy.

This chapter attempts to walk the reader through both a realistic depiction of the project lifecycle as well as some of the associated ML System Design parts (with more detailed description in the rest of the book). Sections 2.2 through 2.5 follow the project lifecycle: how to frame the problem, prepare data, engineer features, and build models for each stage. Starting with Section 2.3, we focus on the system architecture: the multi-stage pipeline that actually serves recommendations.

2.2 Define the Problem

Problem framing is the single most impactful upfront set of decisions that can be made in any Engineering project. In an ML project, this is what determines your downstream

design choices: what data you collect, how you label it, which metrics you optimize, and what tradeoffs are acceptable. Fall in love with the problem, not the solution. A poorly framed problem statement can require significant rework to fix. Most common symptoms include the belated realization that the system has been over-engineered¹ in capabilities that later turned out to be never required and under-engineered on components that actually mattered.

Getting time in front of business/product stakeholders can be tricky and so you want to be going in prepared. The key is to not boil the ocean but instead ask the minimal set of questions designed to elicit just the key information required for you to get started on your architecture design.

2.2.1 Define the problem

Every RecSys initiative should start with a clear articulation of the business or customer problem, not the ML problem. In real life, problem statements don't usually come neatly packaged. Often, you might get forwarded an email with a quarterly roadmap or a meeting invite with a larger group of people where this is discussed. The following will walk you through what needs to be discussed in this phase.

“What customer problem are we trying to solve?”

Expected answers are along the lines of:

- **Discovery:** “Users cannot find relevant content in our large catalog” (e.g., Streaming)
- **Engagement:** “We want users to spend more time engaged on the platform” (e.g., Social media)
- **Conversion:** “We want to increase purchase rates for browsing users” (e.g., e-commerce)
- **Retention:** “We want to reduce churn by surfacing content that keeps users coming back” (e.g., subscription services)
- **Revenue:** “We want to maximize ad revenue per session” (e.g., ad-supported platforms)

A red flag is if the discussion veers off into the solution space such as “We want to replace legacy solution with ML/AI” or “We need more features in the embeddings model”. Resist the temptation to propose solutions immediately, this is often not the

¹ An exception is new product ideas or in early-stage startups where product market fit is uncertain. In those situations, frame the product specs as ‘hypothesis’ on what we **believe** the customer wants.

right forum to do so, nor do you have all the pieces of the puzzle. After letting the stakeholders discuss the problem statement, ask

“What constitutes success?”

A mature product/business team will have a ready answer consisting of a rigorous suite of North Star metric, proxy metrics and guardrail metrics. If you sense that there is additional work to be done here to even define the metrics, defer the alignment on the success metric to a later conversation after you understand the product better. Success metrics is often a surprisingly contentious topic best handled with some preparation on your part.

It is critical for all the stakeholders to be very clear if this project is meant to

- a) Solving a specific customer pain point vs
- b) Continuous improvement of a business metric (e.g., “Improve conversions by X%”).

While both situations may use the same North star metric (e.g., conversions), the purpose is different and this influences how you go about solving it.

The latter situation is far more straightforward and has a broader scope because you can often use any means at your disposal to achieve an X% improvement. For example, reducing latency by 100ms often gives huge improvements to your business metric without doing any model accuracy improvement.

However, if you are asked to solve a specific customer pain point, this is often very tricky for a variety of reasons. One of them being there is a chance that even if you solve it, there is no guarantee that the overall success metric will improve. This often crops up when the issue is only experienced by a very tiny fraction of the user base. And the sensitivity to your North Star metric is minimal.

For example, let's say that the business team states that “*..the recommender carousel is suggesting incompatible products resulting in a poor customer experience*”. They might show you specific examples of the existing recommendations carousel where a wrong sized phone case is being suggested for Iphone16. In such cases, rather than taking this perceived problem statement at face value, gently push back and ask them to quantify the extent of the problem (i.e., no. of customers impacted). Ideally you want the product team to be scoring all the problems with (impact on each customer x no. of customers impacted). If they don't have² any numbers that convey the extensiveness of

² Orgs which are new to integrating with ML models might let their enthusiasm run ahead of the need to define business problem. Or more commonly, bring you niche problems where the AI ROI is negative.

the problem, then work with them to either do the analytics³ (e.g., prompt an LLM to identify incompatible items) or brainstorm any downstream impact to any metric (which hopefully can be quantified). For example, product return rate might capture a percentage of the customers who accidentally bought incompatible products and now want to return it. The metric will also capture other reasons such as returns due to damaged goods. Now try to quantify⁴, how much of the product return rate can be improved if you have a perfectly accurate recommender. If the business value from even a 0% return rate is not large enough, relative⁵ to the (dev effort needed + maintenance cost long term), then it's not worth solving the problem.

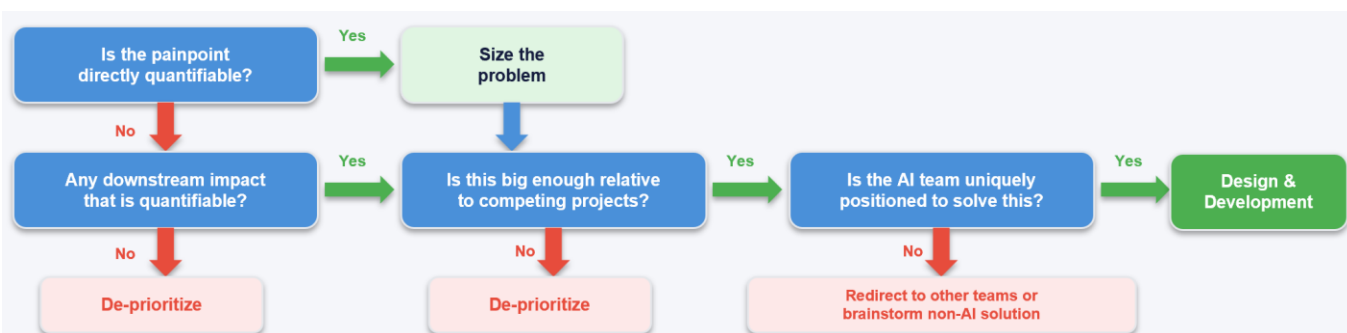


Figure 2.1: Decision framework for evaluating whether a customer painpoint warrants an AI project. The tree begins by asking whether the painpoint is directly quantifiable. If yes, the next step is to size the problem. If the painpoint is not directly quantifiable, the follow-up question asks whether there is any downstream impact that is quantifiable; if not, the painpoint should be de-prioritized. Once sized, the practitioner evaluates whether the problem is large enough to justify investment relative to competing projects. If not, de-prioritize. If yes, the final gate asks whether the AI team is uniquely positioned to solve this compared to other technology teams in the company. A "yes" leads to the design and development phase; a "no" suggests redirecting the request to other teams or brainstorming a non-AI solution.

2.2.2 Understand the user and the product

Before having subsequent meetings with your business team, you need to understand how users actually interact with your product/product surface. Once you are ramped up on your customers and product, you will gain enough trust that the business team will speak to you on equal terms. Key questions and their implications are noted below:

³ Be very clear that you are doing this analytics task to quantify the problem statement rather than dev work towards the final solution. Otherwise, any coding work is often misconstrued as a signal that you have committed to the project!

⁴ This approach is a scaled down version analogous to how MBA classes teach market sizing using Total Addressable Market (TAM), Serviceable Addressable Market (SAM), etc.

⁵ Justifying the ROI of projects, even if no one is asking for it, is a good exercise to level up in your job.

Sample questions	Examples	What ML design choices are influenced by this?
How do users arrive?	Through search, browsing, push notifications, external links.	Feature ideation stage
What page types/ surfaces can users interact?	Home page, product page, etc.	Identifying sources label sources from clickstream event logs
What do we want the user to do after seeing the recommendation? (i.e., call-to-actions ⁶ (CTA))	Click, Add-to-cart, save-to-wishlists, etc.	Proxy metrics and model labelling choices
What currently exists as the baseline?	e.g., sorting by popularity /chronology.	Performance benchmarks to beat
Can the user express their dislike ⁷ ? How often do they do so?	thumbs down, dismiss Ad, etc.	Auxiliary signals for supervised labelling (e.g., Multi-task learning)
In a typical session, what is the interaction density?	Hundreds of interacted items (social media); dozens (food delivery app).	Feature engineering and model selection choices (e.g., RNN)
Do users expect recent interactions to immediately influence subsequent results?	Likely yes for social media reels surfaces.	Data ingestion streaming config
Do different user segments (geography/ language) need to see different results? What is the relative proportion?	Streaming platforms with different catalog per country.	Single model vs multiple finetuned models.
What fraction of your users and items are new on any given day? (cold start)	1% new users. 1 million new content being uploaded (social media)	Influences evaluation metrics and considerations for diversity/novelty.

⁶ This is where you start gaining 'product sense'. As an ML Engineer, you don't necessarily need to read a book on design thinking or take college semester on product development. A couple of sessions with frontend engineering/UX teams is often just about sufficient to understand their lingo.

⁷ In many platforms, users can also report/flag content. This is beyond the scope of the book.

2.2.3 Identify North Star Metrics, Proxy Metrics, and Guardrail Metrics

Assuming these are not already provided, aim to get a shared understanding of the metrics involved. Metrics are the language of alignment between your ML team, any intermediate product team, and your leadership/business stakeholders. So, spending time upfront will save a lot of coordination challenges later. It is common for orgs to organize them into three⁸ tiers:

Tier 1: North Star Metrics. These are what your executive team cares about and the expected improvement in this is what ultimately justifies the upfront investment in any project. They typically are influenced by many factors beyond your system. Examples include:

- Revenue per user
- Monthly active users (MAU) or daily active users (DAU)
- Subscriber retention rate / churn rate
- Customer lifetime value (LTV)

Tier 2: Proxy Metrics. These are the metrics that your ML team directly optimizes. They should be both measurable in the short term and correlated in the long-term with business outcomes (i.e., leading⁹ indicators of Tier 1 metrics). Examples include:

- Click-through rate (CTR)
- Conversion rate (CVR)
- Watch time or listen time
- Items added to cart
- Engagement rate (likes, shares, comments)

Tier 3: Guardrail Metrics. These are metrics that should not degrade when you optimize for Tier 2 proxies. They represent constraints or values that the business is not willing to sacrifice (or only willing to tolerate a small % degradation). Examples include:

- Content diversity - are we showing a variety of categories, not just one?
- Coverage - what fraction of the catalog is ever recommended?
- Fairness metrics - are certain user segments systematically disadvantaged?
- Churn rate – unsubscribes, product returns, etc.

⁸ GenAI tools like Copilot will give a lot more than what is listed here. Try to avoid the temptation of having too many metrics, otherwise prioritization can become cumbersome.

⁹ Make sure to do causal analysis to prove that these are actually leading indicators of Tier 1 metrics!

Usually, business teams are going to start off with Tier 1 metrics. It's usually the Tech lead's job to see how they can be translated to Tier2 metrics that can actually be optimized for. In most product use cases, translating Tier 1 to Tier 2 requires extensive discussions¹⁰ because there is often no one single right answer.

For example, if your use case is a home page video recommender and the Tier 1 business metrics is to increase the visit frequency of the user base. If the line of causality is that by keeping the users engaged in the platform, they come back more often: you could increase the engagement in multiple ways. A simple framing to clarify might be to ask whether they would prefer the users to a) increase the watch time (regression) vs. increase the likes (classification) or some other more nuanced ones like reshares, content creator follows, saves, etc. In practice, if you give them these ideas, they often come back and say "all of the above"! In which case, you will probably need a model that can optimize multiple objectives at the same time.

A common anti-pattern is optimizing aggressively for a Tier 2 proxy (say, CTR) without monitoring guardrails. This can lead to a recommendation system that surfaces clickbait content with long term degradation of business metrics.

Author's Note: Not distinguishing between long term business metric vs short term proxy metric is one of the biggest sources of confusion and disagreement between ML engineers and stakeholders. ML engineers tend to immediately jump to assuming abstract model metrics (NDCG, AUC-PR, Precision/Recall) that are easy to measure offline during model training.

Business stakeholders often go to the other extreme and only talk about very distant long term business metrics that are influenced by many factors beyond the model. Neither are fully wrong, but they are missing the pieces of the puzzle. For various reasons such as the model training data being too sparse/feedback signals being extremely delayed, etc. directly modelling on the long term business metric is difficult. On the other hand, business stakeholders' eyes may glaze over when you start talking abstract math formulas for NDCG without showing them how its related to the customer engagement and why its important to evaluate it. A middle ground is needed.

The three-tier framework helps bridge this gap: you commit to their Tier 1 KPIs as the "why," align on Tier 2 proxies as the "what we optimize", while at the same time tie the proxy metrics to model metrics and set Tier 3 guardrails as the "what we protect."

¹⁰ A related concept is the Overall Evaluation Criteria (OEC). Refer to Ronny Kohavi's excellent book "Trustworthy Online Controlled Experiments"

2.2.4 Non-Functional Requirements (NFRs)

These are typically engineering characteristics of the proposed system that don't directly influence the 'accuracy'.

A few common NFRs and their design implications are given below:

Attribute	Example NFR	Design implications
What is the catalog ¹¹ size? How fast does it grow?	1 Billion items. With 1% growth	Determines how many stages you will need
What is the size of the active user population?	100 million Daily Active Users (DAU); 1 billion Monthly Active Users (MAU)	Determines the size of your user embeddings tables (see chapter 11).
What is the throughput?	10K RPS for home page, 12K for Search, etc.	Influences distributed systems architecture
What is the latency budget for the overall system?	p95 = 200 ms	Influences a lot of things including the size of the model, features being used, choice of tooling, caching, etc.

The last row in the table above assumes that this is a realtime system. However, not every recommendation needs to be computed in real time. The choice between batch and real-time inference depends on how often the recommendations need to change.

Batch inference: Pre-compute recommendations for all users on a schedule (e.g., daily). Suitable for email campaigns, push notifications, homepage modules that are updated daily. Simpler infrastructure, lower cost, but cannot react to real-time signals.

Real-time inference: Compute recommendations on every user request using the latest features. Required for session-aware recommendations, search-triggered recommendations, and any context where the user's current behavior should influence results. More complex infrastructure (feature store, model serving, low-latency pipelines) but provides the best user experience.

¹¹ You likely don't need a multi-stage recsys if your catalog size is less than a few 1000 items. A single stage model should be fast enough to predict within the latency budget.

Where numbers are certain, these will go into the NFRs. For some of the speculative requirements, it's acceptable to take an upper estimate initially and then go back to re-negotiate if this causes issues. The author is a big fan of Donald Knuth's quote, "Premature optimization is the root of all evil"¹². The approach that mitigates over-engineering is to first build a functional MVP¹³, and then iteratively layer in more complexity.

Once we have a good understanding of the project requirements, it is time to start designing the architecture. While all of these are important, the combination of the catalog size, throughput and latency NFRs inevitably leads to a multi-stage recommender system.

Author's note: When teams are building end-to-end systems for the first time, should ML Engineers just focus on getting the functionality right and ignore the NFR aspects initially? Or account for all constraints imposed by the NFRs from day 1?

This is a hotly debated topic and strictly following one or the other perspective can lead to issues later. A team that focuses on NFRs can end up never reaching the finish line on time. And in small organizations, missing deadlines can be the death knell for the project. On the other hand, racing through model development by completely ignoring NFRs results in teams adopting ML design choices fundamentally incompatible with one or more of the NFR constraints (usually latency and costs). Getting the latency wrong by a significant amount will lead to throwaway work. The author's suggestion is to consider NFRs but be very cognizant of the deadlines + risk of over-engineering.

2.3 Multi-stage Recommender System Design

Multi-stage recommender system is the architectural pattern that dominates production recommender systems at virtually every major tech company. The core insight is simple: you cannot score every item in a billion-item catalog with an expensive model within a 50ms latency budget. Instead, you use a cascade of progressively more expensive but more accurate stages.

Several industry practitioners have documented variations of this architecture. Yan (2021) provides a system design framework for recommendation and search pipelines. Oldridge (2022) emphasizes that production recommender systems are full systems, not just models. Amatriain (2023) offers a comprehensive blueprint of recommender system architectures spanning a decade of evolution. Vorotilov and Shugaepov (2023)

¹² The full quote from 1974 is "*The real problem is that programmers have spent far too much time worrying about efficiency in the wrong places and at the wrong times; premature optimization is the root of all evil (or at least most of it) in programming.*"

¹³ Sounds obvious but what usually happens is closer to the launch date, even previously nimble teams start getting cold feet and start tacking on product features that were not in scope originally. Often couched in the language of "We want to put the best foot forward".

describe how Instagram scaled their Explore recommendations pipeline. Agrawal et al. (2024) detail LinkedIn's People You May Know system as a large-scale recommendation application. The architecture described below synthesizes patterns common across these references.

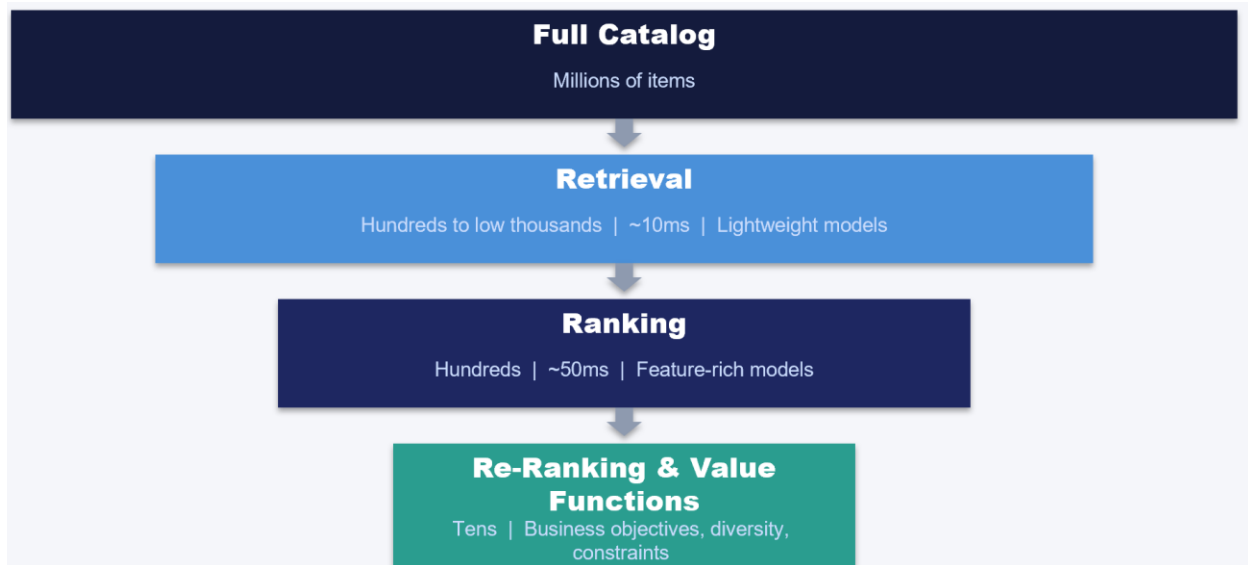


Figure 2.2: The multi-stage recommendation serving funnel. The full item catalog (potentially millions of items) is progressively narrowed at each stage. The retrieval stage reduces the set to hundreds or low thousands using lightweight models within roughly 10 milliseconds. The ranking stage scores these candidates with feature-rich models, reducing to hundreds within roughly 50 milliseconds. The re-ranking and value function stage applies business objectives, diversity constraints, and final adjustments to produce the tens of items actually shown to the user.

2.3.1 Candidate Retrieval & Filtering

The retrieval stage is the top of the funnel. Its job is to quickly narrow the full catalog (potentially millions of items) down to a manageable set of candidates (typically hundreds to low thousands) to pass it on to the subsequent ranking stage. The output items need to be plausibly relevant to the user. Retrieval stage have the following characteristics:

- **Optimize Speed over precision:** retrieval models are deliberately simple to enable fast scoring or ANN lookup.
- **Incorporate Multiple retrieval sources:** production systems often blend candidates from several retrievers - popularity-based, collaborative filtering, embedding similarity, recently interacted, trending, etc.

The retrieval stage sets the ceiling for the entire system's recall. If a relevant item is not retrieved, no amount of sophisticated ranking in the downstream stages can surface it.

Common retrieval approaches include:

- Bi-encoder for an item-item or a Two-tower neural models that encode user and item into a shared embedding space (Chapter 5)
- Popularity and trending baselines (always include these as fallback and cold-start coverage)
- Item-to-item collaborative filtering (“users who liked X also liked Y”)
- Content-based retrieval using item embeddings (Chapter 10)

In Chapter 5, we implement retrieval on the SIGIR E-commerce (Coveo) dataset, a dataset of 48K products and 36 million browsing events.

Between the output of the vector search step and the input to the ranking stage, there is often business logic that handles hard filtering: removing items that should never be shown to the user. Common filters include items the user has already interacted with (to avoid repetitive recommendations), items that violate geographic or legal constraints, and items flagged for content safety. For the “already seen” filter, Bloom filters (Bloom, 1970) are a practical choice. A Bloom filter is a probabilistic data structure that can check set membership in constant time and constant memory, with a small false positive rate but zero false negatives. Each user’s interaction history can be encoded as a Bloom filter, allowing the retrieval stage to cheaply discard items the user has already seen without maintaining an exact set of every past interaction. At scale, where users may have interacted with tens of thousands of items, the space savings over storing exact sets become significant.

2.3.2 Ranking

The ranking stage takes the candidates from retrieval and scores each one using a feature-rich model. This is where the heavy lifting happens: the ranking model has access to detailed user features, item features, contextual features, and user-item interaction features that would be too expensive to have been included in the Retrieval stage.

Key characteristics of ranking:

- **Accuracy over speed** (but still within latency budget, typically 50ms for this stage)
- **Rich feature access:** cross-features, historical aggregates, embeddings

Common ranking approaches include:

- **Gradient boosted trees (e.g., XGBoost):** These were quite prevalent in production ranking in the early 2010s until they were superseded by Deep

Learning (DL) - based ones. In Chapter 6, we train XGBoost on the Yandex Yambda dataset with 50M interactions.

- **Neural ranking models:** DCN-V2 (cross-network for feature interactions), DLRM (Facebook's recommendation model), Wide & Deep, DeepFM. In Chapter 7, we implement DCN-V2, DLRM, and MMoE on Yambda.

If your company is just starting out with ranking models, the initial choice between non-DL vs. DL models is not always clear-cut (Grinsztajn et al., 2022). Trees are more interpretable, train faster, and handle tabular features naturally. Also, popular libraries like dmlc have implemented fairly easy-to-use versions of XGBoost for different use cases (such as point-wise vs pair wise Learning-to-rank approaches). On the other hand, DL models can learn complex feature interactions, scale to more features and easy to extend for multi-task learning. On the flip side, they require more data to learn from and have additional ML engineering effort to host it within a latency budget.

A catalog size of a few 100 million+ often motivates separate stages for light vs heavy ranker models. A faster model like XGBoost can be in the lighter stage while slower heavier neural net models can be in the heavier stage.

The light ranker (sometimes called a pre-ranker or first-pass ranker) operates on the full set of retrieved candidates, which may number in the low thousands. Because it must score every candidate within a tight latency budget, it typically uses a simpler model with fewer features. Gradient-boosted trees, logistic regression, or shallow neural networks with a small feature set are common choices. The light ranker's job is not to produce the final ordering but to eliminate the clearly irrelevant candidates and reduce the set to a few hundred items that warrant deeper evaluation.

The heavy ranker then scores this reduced set using a more expressive model with access to richer features: cross-features between user and item, real-time session signals, embedding similarities, and historical engagement aggregates. Neural architectures like DCN-V2 or DLRM (covered in Chapter 7) are typical heavy rankers. Because the candidate set is already small, the heavy ranker can afford the additional computation. This two-stage ranking approach lets the system achieve both broad coverage (via the light ranker's speed) and high precision (via the heavy ranker's expressiveness) within the overall latency budget.

2.3.3 Re-Ranking & Value Functions

After the ranking model produces a scored list, the re-ranking stage adjusts the final ordering to account for business objectives, diversity, and constraints that a single ranking score cannot capture. Key components of re-ranking include:

Value Functions: When your ranking model predicts multiple objectives (probability of click, probability of like, expected watch time, probability of skip), you need a way to

combine them into a single score. This is the value function which in its simplest form comprises of a weighted linear combination that reflects business priorities:

$$\begin{aligned} \text{Value} = & w_{\text{listen}} \times P(\text{listen}) + w_{\text{like}} \times P(\text{listen}) \times \\ & P(\text{like}|\text{listen}) + w_{\text{engagement}} \times P(\text{listen}) \times E[\text{play_ratio}] - \\ & w_{\text{dislike}} \times P(\text{listen}) \times P(\text{dislike}|\text{listen}) \end{aligned}$$

The weights are tunable business parameters, not learned from data. Changing them lets you shift the system's behavior without retraining the model. In Chapter 8, we implement this on top of the MMoE model from Chapter 7.

Diversity Optimization: Depending on the surface, users may not want to see ten similar items in a row, even if the ranking model scores them all highly. Maximal Marginal Relevance (MMR) and other diversification algorithms balance relevance with diversity. In Chapter 8, we implement MMR using item embeddings to measure intra-list similarity.

Business Rules: Constraints like “no more than 3 items from the same brand in the top 10,” “promoted items in specific slots,” or “recently viewed items excluded.” In Chapter 8, we implement artist pacing for music recommendations, ensuring no artist appears more than once in a short sequence.

2.3.4 Overall system config

While not exactly a stage in the serving pipeline, you need a set of governance rules to let the product/business team control how the entire system behaves. You can think of it as a kind of ‘hyper-parameters’ for the overall system. A few examples are given below:

- **Eligibility rules:** Which candidates are eligible for this user? (filters for geography, language, content safety, age restrictions are applied before the retrieval stage)
- **Retrieval fusion strategy:** Which combination of retrievers should be used?
- **Fallback strategy:** What happens when the primary model times out or returns too few results? (e.g., graceful degradation to popularity-based recs)
- **Model selection for power users:** In mature systems, different models may serve different user segments (e.g., a simpler popularity-based model for new users, a richer model for power users)

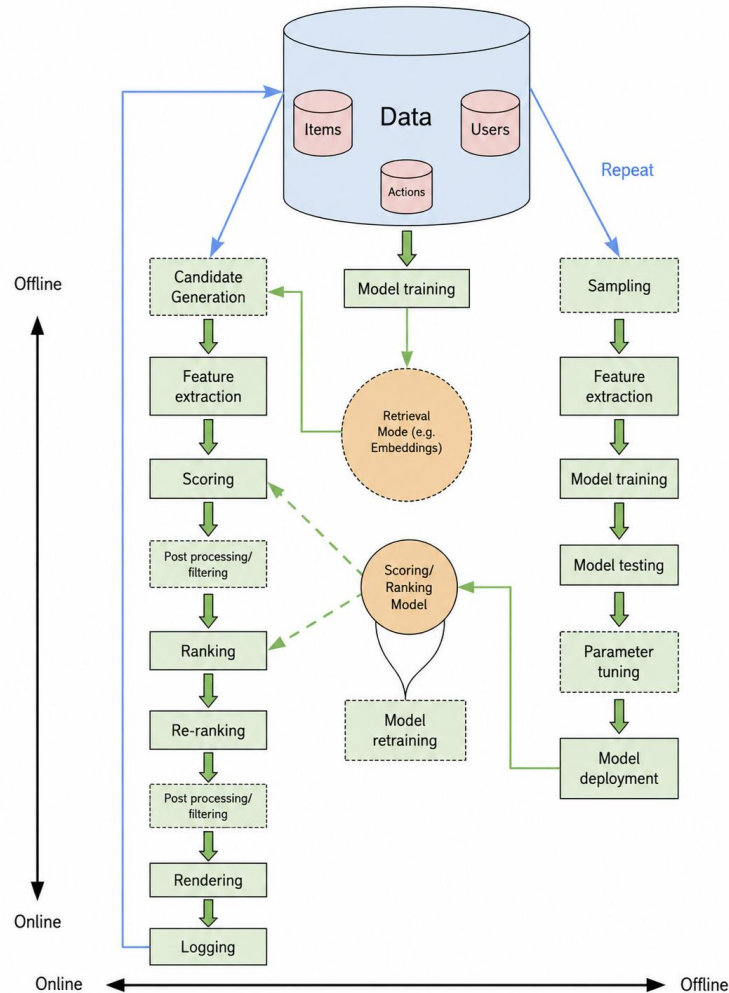


Figure 2.3 Detailed recommender system architecture (adapted from Amatriain, 2023). The online serving pipeline (left) flows top to bottom: candidate retrieval, feature extraction, light ranker scoring (using the retrieval/embedding model), post-processing and filtering, heavy ranker scoring (using the full ranking model), a second round of post-processing and filtering, rendering, and logging. The offline training pipeline (right) covers sampling from the data store, feature extraction, model training, testing, parameter tuning, and deployment of models into the online path. A shared data store at the top (items, users, and actions) feeds both pipelines. Logging output flows back into the data store, completing the retraining feedback loop

2.4 Supervised Learning Data Setup

With the architecture in mind, let us now look at how to set up the supervised learning problem that powers the ranking stage. This is where many of the RecSys-specific subtleties emerge. We will first start off with the labels and then move on to the features.

2.4.1 Obtain the Training set

To ground this discussion, let us look at what a training dataset actually looks like for a recommender system. You will often be given historical user sequence history from which you will need to derive the training set. Using the Yandex Yambda music

recommendation dataset (which we use extensively in Chapters 6-7), a training example might look like this:

user_id	track_id	query_id	label	user_age	track_genre	popularity	hour	position
u_101	t_42	q_7	1	25-34	rock	0.85	14	1
u_101	t_99	q_7	0	25-34	pop	0.32	14	2
u_203	t_42	q_12	1	18-24	rock	0.85	21	1
u_203	t_55	q_12	1	18-24	jazz	0.61	21	3

Each row represents a single user-item interaction opportunity. The label captures whether the user engaged with the item (clicked, listened, purchased, etc.). The features capture everything we know about the user, item, and context at the time of the interaction.

Key observations:

- **The query_id groups interactions that were shown together** (e.g., a playlist or a search results page). This grouping is essential for pairwise and listwise learning-to-rank formulations.
- **Negative examples (label=0) must be handled carefully** — we discuss this in Chapter 3.
- **The position_in_playlist feature is a source of bias** — items in higher positions get more exposure. This is the position bias problem discussed in Chapter 3.

2.4.2 Label Historical Clickstream Data

Turning raw user behavior into training labels is an art. Unlike many other ML domains where labels are provided directly, modern recsys does not solely rely on explicit feedback (“I liked this”). Instead, you have implicit signals (clicks, dwell time, purchases) that are noisy proxies for true preference.

Common labeling strategies include:

- **Binary labels:** Did the user click/purchase?
- **Graded labels:** Map different engagement levels to scores (e.g., click=1, add-to-cart=2, purchase=3). This works well with NDCG-based evaluation.
- **Dwell time thresholds:** Label as positive only if the user spent more than X seconds. (But beware: a user may leave a video playing in the background.)

- **Multi-signal combination:** Combine clicks, dwell time, and downstream actions into a composite label. This is often the most robust approach.

In our music recommendation tutorials (Chapters 6-7), we use listening events as the primary label, with additional signals for likes, play ratio, and dislikes feeding into the multi-task MMoE model.

Recsys datasets are usually time series datasets. In many domains, the user's engagement with a recommended item is not immediate. A user might browse a product today and purchase it in three days. If your training label only captures same-session interactions, you will miss these delayed conversions.

Key considerations:

- **Attribution window:** How long after a recommendation do you attribute a conversion to it? Same session? 24 hours? 7 days?
- **Label maturity:** You may need to wait days or weeks before your labels are “complete” — meaning all delayed conversions have been captured. This creates a lag between data collection and model training.
- **The tension between freshness and label quality:** shorter windows give you fresher training data but noisier labels (missing delayed conversions); longer windows give better labels but staler data.

This is particularly acute in adtech, where a user might click an ad, leave the site, and convert days later. Even when we correctly account for the time difference, there are situations where the training records used is polluted by selection bias. In Chapter 9, we address this with the ESMM architecture, which explicitly models the conversion funnel (impression → click → conversion) to handle selection bias.

2.4.3 Formulate as Pointwise, Pairwise, and Listwise

The ranking problem can be formulated in three fundamentally different ways and these formulations comes from the Information Retrieval (IR) community. While IR focuses on ranking search results for a query, the same mathematical frameworks apply directly to ranking recommendations for a user.

Pointwise: Treat each item independently. Predict a score (probability of click, expected watch time) for each item, then sort by score. This is the simplest formulation and works surprisingly well. XGBoost with binary classification or regression is a pointwise approach.

Pairwise: Instead of predicting absolute scores, predict which of two items should rank higher. The model learns from pairs of items (one relevant, one not) within the same query context. RankNet (Burges et al., 2005) and LambdaMART (Burges, 2010) use

this formulation. The advantage is that rather than absolute scores, the model directly optimizes the ordering which is what matters for ranking problems outside Adtech.

Listwise: Optimize the entire ranked list against a metric like NDCG. This directly optimizes what we care about but is computationally more expensive and harder to implement. Approaches like LambdaRank and softmax-based losses fall into this category.

2.4.5 Feature Ideation

Features are the raw material of your ranking model. The quality of your features matters far more than the sophistication of your model architecture. When you are creating supervised ML models, a clean way to organize your feature space is to segment them into 3 Cs – Content, Customer and Context

Content (Item) Features: Everything about the item that is independent of the specific user.

- Metadata: category, brand, price, description, tags
- Popularity signals: number of interactions in last 24h/7d/30d, trending score
- Quality signals: average rating, review count, return rate
- Item embeddings: learned representations from Chapter 10

Customer (User) Features: Everything about the user that is independent of the specific item being scored.

- Demographics: age bucket, gender, location, language
- Behavioral aggregates: total interactions in last 7/30/90 days, category preferences, average session length
- Engagement patterns: time-of-day preferences, device preferences, frequency of visits
- User embeddings: learned representations from Chapter 11

Context Features: Situational factors that change with each request.

- Time: hour of day, day of week, season, holidays
- Device: mobile vs. desktop, screen size, operating system
- Location: country, city, timezone
- Session state: items already viewed in this session, items in cart, search query (if any)

This grouping of features also enables conversations on which features can be precomputed (customer and content features) versus which must be computed at request time (context features and cross-features). This distinction has direct implications for your feature pipeline architecture (Chapter 4).

As your organization becomes more mature and the data engineering pipelines more stable, it is also worthwhile to ask for User-Item features (or Query-Item features if you are working on Search problems).

2.4.6 Feature ‘Selection’

Given this plethora of feature possibilities, it’s helpful to prioritize which are likely to have the biggest impact. Usually, user’s past behavior and item content features are most predictive. If this is your first project, it is highly suggested to create a spreadsheet to list out all the feature ideas in each ‘group’, then figure out if the features are already available or need to be data engineered. This usually results in a table like so

Table 2.1: Prioritization of feature engineering

Feature Group	Feature idea	Actual feature name	Available offline?	Available online ¹⁴ ?	Priority
User Item interaction	Last 24 hours like counts	User_daily_total_likes in Enterprise.user_table	Yes	Yes	High
User Item interaction	Last 24 hours search terms	Does not exist. Raw features in Enterprise.Search_metadata	tbd	tbd	High
User Item interaction	Last 6 months product returns	Does not exist	Tbd	Tbd	Low
Item metadata	Last 24 hours total clicks	item_daily_total_clicks in Enterprise.content table	Yes	Yes	High

¹⁴ We will further discuss the varying needs of offline vs online features in chapter 4

In an ideal world, you would have all the features readily available in a clean enough format in both offline and online low latency consumption data stores. Then all that the model owner needs to do is throw them all in the model and let it learn from the data on which features are important to be included.

In reality, the sheer volume of feature ideas can make the data engineering dev effort non-trivial. You might be thinking to run a feature selection and keep the top ones. However, to even do algorithmic feature selection, you will need to create the features and join with the training set.

If ML and data teams are new to working with each other, there can be substantial differences in the granularity of the data needed for model training vs what is available. Hence, its often the case that the project lead makes a subjective call on what features to prioritize for data engineering for the initial launch vs later iterations. And after the data engineering is done, there is a further round of 'feature selection' through usually a round of data analysis (for example, by looking at the feature sparsity and discarding features having more than 95% nulls). After a few such projects, the team will get an intuitive sense of which features will likely be of the highest data quality and usefulness to the model.

And finally more features does not automatically guarantee that the resulting model will be substantially better: the author was working with a team of Research scientists to rebuild a legacy tree-based model that incorporated 600+ features (most of them heavily feature engineered variations such as `feature_name_30d`, `feature_name_60d`, etc.). Based on understanding the domain, the author proposed a hand-curated list of ~20 features that they believed to be important fed into a neural net-based sequential¹⁵ model. The new model achieved parity performance in offline metrics. The takeaway was that a small group of highly curated features can have a very high signal-to-noise ratio and can allow highly accurate yet lightweight models to be built.

2.4.7 Mitigating Exposure Bias and Feedback Loops

One of the most insidious problems in RecSys feature engineering is that your training data is biased by the previous recommendation system. Users only interact with items they were shown based on the old model's predictions. If the old system had a popularity bias, this often manifests itself as:

¹⁵ Moving to a sequence model like RNN also removes the need to do extensive feature engineering of time-series features.

- Popular items being over-represented in your training data (because they were shown more often).
- Items in higher positions receiving more engagement (position bias) regardless of relevance.
- User preferences appear narrower than they actually are (because the old model never showed diverse content enough times for it to catch on with the customers).

Mitigation strategies include:

- **Inverse propensity scoring (Joachims et al., 2017):** Re-weight training examples by the inverse of their probability of being shown. Items that were shown rarely but still generated engagement get higher weight.
- **Position features:** Include the position at which an item was shown as a feature during training. At serving time, set this feature to a constant (e.g., position 1) so the model predicts engagement independent of position.
- **Exploration data:** Reserve a small percentage of traffic (1-5%) for random or diversified recommendations. This unbiased data is extremely valuable for evaluation and model correction.

We won't go into these topics in more detail, but the users are encouraged to check out the literature on these.

2.4.8 Encode the features

Once you have identified features you are going to include in your model, they need to be encoded into a format that your model can consume. The type of feature encoding will depend on the feature type, the specific data distribution and the type of model that will be using them. While its not possible to give an exhaustive list of all possible encoding, listed below are some common ones:

Continuous numeric features (price, rating, counts, durations) should be normalized or bucketed. Common approaches:

- Log transformation for skewed distributions (e.g., view counts)
- Quantile bucketing for heavy-tailed distributions
- Min-max or z-score normalization

Categorical features (genre, brand, device type) need to be encoded as numbers:

- One-hot encoding for low-cardinality features (< 50 categories)
- Target encoding or frequency encoding for medium-cardinality

- Hashed encoding for high-cardinality features (user IDs, item IDs)
- Learned embeddings for very high cardinality (for neural models like DLRM)

Text features (titles, descriptions, reviews) can be encoded as:

- TF-IDF vectors (simple, interpretable baseline)
- Pre-trained sentence embeddings (SBERT, as used in Chapter 10)
- Domain-specific embeddings (e.g., RexBERT for e-commerce, as covered in the Appendix)

Image and audio features are typically encoded using pre-trained deep networks (ResNet for images, VGGish or i-vectors for audio).

Table 2.2 Feature Encoding Summary

Feature Type	Low-Complexity Approach	High-Complexity Approach	When to Upgrade
Continuous	Bucketing, log transform	Quantile normalization	Skewed distributions
Categorical (low card)	One-hot encoding	Frequency encoding	When categories are sparse
Categorical (high card)	Hashed encoding	Learned embeddings	Neural models, >1000 categories
Text	TF-IDF	Pre-trained sentence embeddings	When text is the primary signal
Image/Audio	Pre-trained CNN features	Fine-tuned domain embeddings	Multimodal systems
Cross-features	Manual combinations	DCN-V2 cross-network	When interactions are critical

Between feature engineering and deployment sits the modelling and offline evaluation phase: selecting model architectures, training on the labeled dataset, tuning hyperparameters, and evaluating candidates against held-out data using ranking metrics such as NDCG, MRR, and AUC. These phases are covered extensively in the chapters that follow. Chapter 3 lays out the evaluation methodology and common pitfalls, while Chapters 5 through 12 each walk through model training and evaluation for their respective pipeline stages. Rather than repeat that treatment here, this chapter proceeds to the deployment considerations that complete the project lifecycle.

2.5 Deployment Overview

This section provides a high-level overview of deployment considerations. Detailed treatment is in Chapters 13-15.

2.5.1 Verify functional and non-functional characteristics:

After testing the model in isolation, there are a series of tests that happen depending on the company's engineering practices. At a high level, there needs to be a performance test to verify that the overall system can handle the required throughput under the latency SLA. Companies establish dedicated test harnesses that simulate production traffic load for a few hours and stress¹⁶ test the system.

The output logs from the test are examined to see if the system breaks under the load (often tested at multiple of the actual production traffic). This is often done in parallel with functional tests where a dedicated QA/product team goes through a set of test cases to verify functionality. For example, searching for a 'gaming laptop' should return laptops with GPUs, etc. If you are launching for the first time, as the model owner, you will need to sit down and come up with several test cases for the functional validation and influence your engineering counterparts to building the test harness in a pre-prod environment.

2.5.2 A/B Testing and Evaluation

The gold standard for evaluating an online system is an A/B test: showing different recommendations to randomly assigned user groups and measuring the impact on business metrics. Offline metrics (NDCG, MRR) are necessary for development but insufficient for launch decisions.

Key considerations:

- **Statistical significance:** recommendation experiments often have high variance, requiring large sample sizes.
- **Novelty effects:** users may initially engage more (or less) with new recommendations simply because they are different. Run experiments long enough for this effect to fade.
- **Metric sensitivity:** Tier 1 business metrics may take weeks to show a signal. Tier 2 proxy metrics are more sensitive and can serve as early indicators.

We discuss experimentation in detail in Chapter 16, including interleaving experiments and bandit-based approaches as alternatives to traditional A/B tests.

2.5.3 Monitoring

Once deployed, your recommender system needs continuous monitoring across multiple dimensions:

¹⁶ Some tech companies take it to the extreme – Netflix created Chaos Monkey that randomly terminates VMs to get engineers to build resilient systems (<https://github.com/netflix/chaosmonkey>)

- **System health:** latency, error rates, throughput
- **Data quality:** feature drift, missing values, schema changes
- **Model performance:** online proxy metrics (CTR, engagement), distribution of model scores, calibration drift
- **Business impact:** Tier 1 KPIs tracked against pre-deployment baselines

We cover monitoring and retraining strategies in detail in Chapter 15.

2.6 Author's Notes

In my experience, the most successful RecSys projects share a few characteristics. First, the working group have established a strong connection between offline evaluation and online metrics, the individuals know which offline improvements translate to online gains. Second, there has been heavy investment in testing infrastructure before investing in model complexity. Third, the ML teams start with simple models and strong baselines, and only move to more complex approaches when they have evidence that the simpler models are leaving value on the table. If it's the first time, the company is building a realtime ML systems, the initial few months are spent in debugging things in Prod, with the most typical issues being data quality. Only after you have confidence that your production system is working as intended and has been stable for several weeks without major issues is it worth investing in more advanced modelling frameworks.

The framework in this chapter is meant to accelerate that process. Whether you are building your first recommendation system or improving an existing one, walking through each section systematically will help you identify the decisions that matter most for your specific use case.

2.9 Summary

In this chapter, we developed a comprehensive framework for designing recommender systems:

- We distinguished between the project lifecycle (the sequence of decisions) and the system architecture (the runtime components).
- We covered problem framing, emphasizing business alignment, user interaction patterns, and the three-tier metrics framework (Business KPIs, Proxy Metrics, Guardrails).
- We deconstructed the multi-stage architecture: Retrieval → Ranking → Re-Ranking/Value Functions → Policy Layer.

- We discussed the supervised learning setup: training DataFrame structure, labeling strategies, time windows, and pointwise/pairwise/listwise formulations.
- We organized feature engineering along three dimensions (Customer, Content, Context) and covered encoding strategies.
- We provided a high-level overview of deployment, A/B testing, and monitoring.

In the next chapter, we dive deeper into the data foundation: how to collect, clean, and evaluate the training data that powers your models.

References

1. Agrawal, P., Gupta, V., Gupta, A., & Nigam, A. (2024). Building a Large-Scale Recommendation System: People You May Know. LinkedIn Engineering Blog.
2. Amatriain, X. (2023). Blueprints for Recommender System Architectures: 10th Anniversary Edition. amatria.in.
3. Bloom, B. H. (1970). Space/Time Trade-offs in Hash Coding with Allowable Errors. *Communications of the ACM*, 13(7), 422–426.
4. Burges, C., et al. (2005). Learning to Rank using Gradient Descent. *ICML*.
5. Burges, C. (2010). From RankNet to LambdaRank to LambdaMART: An Overview. Microsoft Research Technical Report MSR-TR-2010-82.
6. Cheng, H.-T., et al. (2016). Wide & Deep Learning for Recommender Systems. *DLRS '16*.
7. Covington, P., Adams, J., & Sargin, E. (2016). Deep Neural Networks for YouTube Recommendations. *RecSys '16*.
8. Grinsztajn, L., Oyallon, E., & Varoquaux, G. (2022). Why Do Tree-Based Models Still Outperform Deep Learning on Tabular Data? *NeurIPS*.
9. Joachims, T., Swaminathan, A., & Schnabel, T. (2017). Unbiased Learning-to-Rank with Biased Feedback. *WSDM*.
10. Ma, J., Zhao, Z., Yi, X., et al. (2018). Modeling Task Relationships in Multi-task Learning with Multi-gate Mixture-of-Experts. *KDD*.
11. Naumov, M., et al. (2019). Deep Learning Recommendation Model for Personalization and Recommendation Systems. *arXiv*.
12. Oldridge, E. (2022). Recommender Systems, Not Just Recommender Models. *NVIDIA Merlin Blog*.
13. Vorotilov, V., & Shugaepov, I. (2023). Scaling the Instagram Explore Recommendations System. *Meta Engineering Blog*.
14. Wang, R., et al. (2021). DCN V2: Improved Deep & Cross Network. *WWW '21*.
15. Yan, E. (2021). System Design for Recommendations and Search. *eugeneyan.com*.